

Praktikumsaufgaben 10

Aufgabe 1 „Verarbeitung von Bestellungen“ – Unit-Test

Anwendungen müssen Zahlen und Transaktionen sicher verarbeiten. Die Klasse `OrderService` bearbeitet Bestellungen und stellt dabei eine Methode bereit: `processOrder()`. In dieser Methode interagiert die Klasse mit einer Schnittstelle `IPaymentGateway`, um Zahlungen zu authentifizieren. Die Schnittstelle besitzt die Methode `processPayment()`.

Erstellen Sie Unit-Tests für die Klasse `OrderService`. Verwenden Sie für die Abhängigkeit `IPaymentGateway` die folgenden Test-Doubles:

1. Stub: Implementieren Sie einen Stub für `IPaymentGateway` und verwenden Sie ihn in den Unit-Tests.
2. Mock ohne Framework: Entwickeln Sie einen Mock für `IPaymentGateway` manuell, d. h. ohne Einsatz eines Mocking-Frameworks.
3. Mock mit Framework: Implementieren Sie die entsprechenden Unit-Tests erneut unter Verwendung eines Mocking-Frameworks.
4. Ermitteln Sie die Code-Abdeckung.

Hier ist der Beispielcode zur Orientierung:

```
public interface IPaymentGateway {
    boolean processPayment(double amount);
}

public class Order {
    private double amount;

    public Order(double amount) {
        this.amount = amount;
    }

    public double getAmount() {
        return amount;
    }
}

public class OrderService {
    private IPaymentGateway paymentGateway;

    public OrderService(IPaymentGateway paymentGateway) {
        this.paymentGateway = paymentGateway;
    }

    public Order processOrder(Order order) {
        if (paymentGateway.processPayment(order.getAmount())) {
            return order; // Bestellung erfolgreich
        }
        return null; // Zahlung fehlgeschlagen
    }
}
```

Aufgabe 2 „Benutzerregistrierung“ – Unit-Test

Eine Anwendung verwaltet die Registrierung von Benutzern. Sie sendet an den Benutzer eine Bestätigungs-E-Mail während des Registrierungsprozesses. Die Klasse `UserService` ist zuständig für die Benutzerregistrierung (Methode `registerUser()`). Sie interagiert mit einer Schnittstelle `IService`, die eine Methode zum Versenden von Bestätigungs-E-Mails beinhaltet: `sendConfirmationEmail()`. Erstellen Sie Unit-Tests für die Klasse `UserService`. Der Test sollte verifizieren, dass die `sendConfirmationEmail`-Methode mit der richtigen E-Mail-Adresse aufgerufen wird. Für die Klasse `IService` verwenden Sie einen Mock **ohne und mit** Einsatz eines Mocking-Frameworks. Ermitteln Sie die Code-Abdeckung.

Hier ist der Beispielcode zur Orientierung:

```
public interface IService {
    void sendConfirmationEmail(String email);
}

public class User {
    private String name;
    private String email;

    public User(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getEmail() {
        return email;
    }
}

public class UserService {
    private IService emailService;

    public UserService(IService emailService) {
        this.emailService = emailService;
    }

    public void registerUser(User user) {
        // Benutzerregistrierung (z. B. in einer Datenbank
        // speichern)
        // Sende Bestätigungs-E-Mail
        emailService.sendConfirmationEmail(user.getEmail());
    }
}
```

Aufgabe 3 „Applikation-Analytics“ – Unit-Test

Die Klasse `AnalyticsService` verfolgt die Nutzung einer Anwendung. Die Methode `trackEvent()` ruft die Methode `logEvent()` der Klasse `LoggingService` auf, um die Ereignisse aufzuzeichnen. Schreiben Sie Unit-Tests für die Klasse `AnalyticsService`. Der Test sollte verifizieren, dass die `trackEvent`-Methode korrekt aufgerufen wird. Für die Klasse `LoggingService` verwenden Sie einen Spy. Ermitteln Sie die Code-Abdeckung.

Hier ist der Beispielcode zur Orientierung:

```
// LoggingService-Interface, die für die Protokollierung
zuständig ist
public interface ILoggingService {
    public void logEvent(String event);
}

public class LoggingService implements ILoggingService {
    public void logEvent(String event) {
        // Protokolliere das Ereignis (z. B. in einer Datenbank)
        System.out.println("Ereignis protokolliert: " + event);
    }
}

// AnalyticsService-Klasse, die die Ereignisse verfolgt
public class AnalyticsService {
    private ILoggingService loggingService;

    public AnalyticsService(ILoggingService loggingService) {
        this.loggingService = loggingService;
    }

    public void trackEvent(String event) {
        // Erledige einige Sachen ... Dann
        loggingService.logEvent(event);
    }
}
```